

Nexsync: A Local-First Cross-Platform Collaborative Platform with Integrated Real-Time Code Editor

Project + Research Paper

Yash Parab & Dhruv Dinda

Department of Computer Science

Somaiya School of Basic & Applied Sciences

Somaiya Vidyavihar University

Vidyavihar - 400 077, Mumbai, Maharashtra, India

yash.sp@somaiya.edu & dhruv.dinda@somaiya.edu

Abstract

Centralized collaboration tools suffer from constant connectivity requirements, privacy risks, vendor lock-in, and poor offline support. Nexsync is a local-first, cross-platform collaborative platform for students and small teams that combines knowledge management with a powerful real-time code editor. All data lives locally on the user's device and works fully offline. Synchronization occurs automatically via Conflict-free Replicated Data Types (CRDTs) when peers connect. The platform supports rich-text notes, Kanban boards, assignment tracking, file attachments, and a syntax-aware real-time code editor that preserves code structure during concurrent edits. Built with Tauri for cross-platform desktop experience and Yjs for CRDT synchronization, Nexsync delivers privacy-preserving, resilient collaboration with seamless real-time and offline capabilities.

Introduction

Problem Statement

Students and small development teams frequently work in environments with unreliable internet, facing issues with centralized tools such as data privacy concerns, service outages, version conflicts, and limited offline functionality. Existing solutions either excel at document collaboration (e.g., Notion) or code editing (e.g., VS Code Live Share) but rarely combine both effectively in a local-first, privacy-focused manner.

Project Introduction

Nexsync addresses these challenges by creating a unified local-first platform. Every device holds a complete copy of shared workspaces. Changes made offline are automatically reconciled using CRDTs upon reconnection. It integrates rich knowledge management features with a syntax-aware collaborative code editor, enabling seamless teamwork on both notes/documents and source code within the same workspace. The system supports peer-to-peer synchronization, end-to-end encryption, and contribution analytics while maintaining a clean, responsive cross-platform experience.

Scope

The project targets individual students, small student teams, and small development groups. It focuses on core collaboration features, offline resilience, and integrated code editing.

In-scope:

- Local-first storage with SQLite and CRDT-based data models.
- Real-time and asynchronous peer-to-peer synchronization.
- Rich-text notes, Kanban boards, assignment tracker, file attachments, and integrated real-time code editor.
- Syntax-aware conflict resolution for code to preserve parseability.
- End-to-end encryption for shared workspaces.
- Cross-platform application for desktop and mobile.
- Contribution analytics and basic version history.

Out-of-scope:

- Large-scale enterprise features or massive user bases.
- Advanced AI integrations or full video/audio collaboration.
- Native mobile apps (focus on desktop + PWA).

Literature Review

1. Almishev (2025) systematizes methods and tools for developing collaborative applications using CRDTs, with emphasis on user conflict resolution, local-first approaches, and libraries like Yjs and Automerge. This provides foundational insights for Nexsync's synchronization strategy.
2. Litt et al. (2022) introduced Peritext, a CRDT for collaborative rich text editing that preserves author intent with formatting spans. This serves as the foundational model extended in Nexsync for both rich-text notes and syntax-aware code structures.
3. Haas et al. (2024) propose a programming model for verifiably safe local-first software, analyzing limitations of existing CRDT frameworks like Yjs and Automerge while offering safety guarantees relevant to educational collaborative tools.
4. A 2025 study on a local-first collaborative modeling approach with replicated data types demonstrates decentralized collaboration patterns that align with Nexsync's offline-first Kanban, notes, and code editing features.
5. Pandey et al. (2025) explore local-first distributed property graphs, addressing consistency, performance, and usability trade-offs in decentralized knowledge management systems.
6. Lehmann et al. (2026) investigate collaborative document editing with multiple users using Yjs for CRDT-based real-time syncing, offering benchmarks for rich-text and structured collaboration in productivity apps.
7. Research on CAMS F Edge DTN (2026) examines context-aware offline-first synchronization architectures, highlighting benefits for unreliable network environments common in student and small-team settings.
8. A Compositional CRDT for Collaborative Spreadsheets (2026) introduces techniques adaptable to structured data like Nexsync's assignment trackers and Kanban boards.
9. Loro.dev contributors (2024–2026) implemented and analyzed rich text CRDTs compliant with Peritext criteria, including Rust implementations, demonstrating practical performance in decentralized collaboration and informing Nexsync's merging strategies for code and text.

10. Comprehensive surveys of the local-first software landscape in 2026 review CRDT libraries, sync engines, and production patterns for offline-capable collaborative applications, including Tauri-based implementations.
11. Ink & Switch Research Collective (ongoing 2025–2026) on Peritext, Pushpin, and related works provide practical lessons in P2P collaboration, rich-text CRDTs, and intent preservation adaptable to code editing.
12. MoonBit CRDT implementations (2026) explore OT, RGA, and advanced CRDT algorithms with collaborative editor demos, offering insights into performant syntax-aware editing.
13. AI agents as CRDT peers with Yjs (2026) demonstrates integrating collaborative editors with real-time systems, relevant for future extensions of Nexsync’s code and knowledge features.
14. Local-first collaborative spreadsheets and Kanban studies (2025–2026) evaluate CRDT usefulness in complex structured editors, validating approaches for Nexsync’s boards and trackers.
15. Industry and academic analyses of Tauri + Yjs + SQLite applications (2025–2026) emphasize cross-platform local-first patterns, data ownership, privacy, and integration best practices for educational and development tools.

Proposed Methodology

System Architecture

- Storage Layer: SQLite for local persistence of documents, boards, code files, and metadata. CRDT structures (Yjs Shared Types) manage collaborative state.
- Synchronization Layer: Yjs CRDTs for automatic conflict-free merging; WebRTC for direct peer-to-peer connections and WebSockets for optional signaling.
- Editor Layers: BlockNote (Yjs-backed) for rich-text notes; integrated CodeMirror/Monaco-based editor with syntax-aware CRDT extensions for code.
- Frontend/Application Layer: Cross-platform UI with rich-text, Kanban, assignment, and code editing components.
- Security Layer: Client-side libsodium for E2E encryption of shared workspaces.
- Core Workflow: Create/edit locally → Automatic CRDT updates → Sync on connectivity → Seamless merges across notes and code.

Technologies to be Used

Technology	Description
Tauri + React (TypeScript)	Cross-platform desktop runtime and frontend
TailwindCSS + shadcn/ui	Modern, responsive UI
Yjs	Primary CRDT library for real-time sync
SQLite	Local-first database
BlockNote + CodeMirror	Rich-text and syntax-aware code editing
WebRTC / WebSockets	Peer-to-peer and real-time synchronization
libsodium	End-to-end encryption

Key Features

- Collaborative rich-text notes with real-time cursors and history.
- Integrated real-time code editor with syntax-aware CRDT conflict resolution.
- Shared Kanban boards and assignment/deadline tracker.
- File attachments with local versioning.
- Full offline-first operation with automatic peer-to-peer sync.
- End-to-end encryption for shared workspaces.
- Contribution analytics, version history, and semantic diffs for code.
- Responsive, modern UI with dark mode and accessibility focus.
- Export options (PDF, Markdown, code snapshots).

Conclusion

Nexsync unifies local-first knowledge management and real-time collaborative coding into a single privacy-preserving platform. By leveraging Yjs CRDTs, Tauri, and integrated editors, it empowers students and small teams with true data ownership, robust offline capabilities, and seamless collaboration across documents and code. The project builds upon established research in CRDTs and local-first systems while demonstrating practical integration of code editing features. Limitations (such as initial peer discovery) are acknowledged, with the modular architecture allowing future enhancements. Nexsync contributes to the local-first ecosystem and offers a strong foundation for next-generation educational and development tools.

References

1. Almishev, K. (2025). Methods and Tools for Developing Collaborative Applications Based on CRDT Types with User Conflict Resolution. Universal Library of Engineering Technology.
https://ulopenaccess.com/papers/ULETE_V02I04/ULETE20250204_015.pdf
2. Litt, G., et al. (2022). Peritext: A CRDT for Collaborative Rich Text Editing. Proceedings of the ACM on Human-Computer Interaction.
<https://dl.acm.org/doi/10.1145/3555644>
3. Haas, J., et al. (2024). A Programming Model for Verifiably Safe Local-first Software. ACM.
<https://dl.acm.org/doi/full/10.1145/3633769>
4. Olivier, L., et al. (2025). A Local-First Collaborative Modeling Approach with Replicated Data Types. HAL.
<https://cea.hal.science/cea-05322894/document>
5. Pandey, A., et al. (2025). Towards Local-First Distributed Property Graphs. ACM.
<https://dl.acm.org/doi/pdf/10.1145/3721473.372213>
6. Lehmann, F., et al. (2026). Collaborative Document Editing with Multiple Users and AI. ACM.
<https://dl.acm.org/doi/full/10.1145/3772318.3790648>
7. [Authors]. (2026). CAMS F Edge DTN: Context-Aware Offline-First Synchronization. MDPI.
<https://www.mdpi.com/1999-5903/18/4/180>
8. [Authors]. (2026). A Compositional CRDT for Collaborative Spreadsheets. ACM.
<https://dl.acm.org/doi/10.1145/3806077.3806695>

9. Loro.dev Team. (2024–2026). crdt-richtext - Rust implementation of Peritext and Fugue. Loro Blog.
<https://loro.dev/blog/crdt-richtext>
10. Various Authors. (2026). Local-First Software: Principles, Patterns, and Technologies.
<https://wal.sh/research/local-first>
11. Ink & Switch Research Collective. (Ongoing 2025–2026). Peritext and Related Works on P2P Collaboration.
<https://www.inkandswitch.com/peritext/>
12. Chen, Z. et al. (2026). Implementing CRDT Algorithms with MoonBit and Building Collaborative Editors. MoonBit Blog.
<https://www.moonbitlang.com/blog/crdt-algorithms-moonbit>
13. Electric SQL Team. (2026). AI agents as CRDT peers — building collaborative AI with Yjs.
<https://electric.ax/blog/2026/04/08/ai-agents-as-crdt-peers-with-yjs>
14. Hatcher, N. et al. (2025–2026). Local first collaborative Spreadsheets. Local First Conf.
<https://app-2026.localfirstconf.com/schedule/day-2/1430-local-first-collaborative-spreadsheets-are-crdts-u>
15. Various Industry & Academic Analyses. (2025–2026). Tauri, Yjs, and Local-First Application Patterns.
<https://tauri.app/blog/>